

SECURE-WEB-LOCK: CLIENT-SIDE FILE ENCRYPTION BERBASIS WEB MENGGUNAKAN AES-GCM 256-BIT DAN PBKDF 2

M. Rizki Juniardi ¹⁾, Muhammad Alfath Mavianza ¹⁾, Muzayyan Rozaan. ¹⁾

1) Mahasiswa Program Studi Teknik Informatika Universitas Mataram

Corresponding author e-mail: fld02310074@student.unram.ac.id

Abstrak

Pertukaran data melalui penyimpanan awan (*cloud storage*) rentan terhadap penyadapan dan kebocoran privasi karena mayoritas penyedia layanan menerapkan *Server-Side Encryption*, di mana kunci dekripsi dikendalikan oleh pihak ketiga. Penelitian ini mengusulkan pengembangan Secure-Web-Lock, sebuah aplikasi pengaman berkas berbasis web dengan arsitektur *Zero-Knowledge* yang menerapkan *Client-Side Encryption*. Aplikasi ini mengeksekusi seluruh operasi kriptografi secara lokal pada peramban pengguna memanfaatkan Web Crypto API, sehingga berkas asli dan kata sandi tidak pernah ditransmisikan ke peladen. Algoritma *Advanced Encryption Standard* (AES) dengan mode *Galois/Counter Mode* (GCM) 256-bit diimplementasikan untuk menjamin kerahasiaan sekaligus integritas berkas melalui *Authentication Tag*. Untuk memitigasi serangan *brute-force*, kunci kriptografi diturunkan menggunakan *Password-Based Key Derivation Function 2* (PBKDF2) dengan fungsi *hash* SHA-256 dan 600.000 iterasi sesuai standar keamanan terkini. Pengujian fungsional menunjukkan bahwa sistem berhasil mengenkripsi dan mendekripsi berbagai format berkas tanpa mengorbankan portabilitas, serta secara absolut mampu menggagalkan upaya dekripsi pada berkas yang telah dimodifikasi secara ilegal (*tampering*).

Kata kunci: AES-GCM; Client-Side Encryption; PBKDF2; Web Crypto API; Zero-Knowledge

Abstract

Data exchange through cloud storage is vulnerable to interception and privacy breaches because the majority of service providers implement *Server-Side Encryption*, where decryption keys are controlled by third parties. This research proposes the development of *Secure-Web-Lock*, a web-based file security application with a *Zero-Knowledge* architecture that implements *Client-Side Encryption*. This application executes all cryptographic operations locally on the user's browser utilizing the Web Crypto API, ensuring that the original files and passwords are never transmitted to the server. The *Advanced Encryption Standard* (AES) algorithm with 256-bit *Galois/Counter Mode* (GCM) is implemented to guarantee file confidentiality and integrity via an *Authentication Tag*. To mitigate *brute-force* attacks, the cryptographic key is derived using *Password-Based Key Derivation Function 2* (PBKDF2) with the SHA-256 hash function and 600,000 iterations in accordance with current security standards. Functional testing demonstrates that the system successfully encrypts and decrypts various file formats without sacrificing portability, and is absolutely capable of thwarting decryption attempts on illegally modified files (*tampering*).

Keyword: AES-GCM; Client-Side Encryption; PBKDF2; Web Crypto API; Zero-Knowledge

I. PENDAHULUAN

Perkembangan teknologi informasi telah mendorong peningkatan aktivitas pertukaran data digital dalam berbagai sektor kehidupan. Layanan penyimpanan awan (*cloud storage*) dan platform berbagi berkas memungkinkan pengguna untuk mengirimkan dokumen dengan cepat dan efisien. Namun, kemudahan tersebut berbanding lurus dengan peningkatan risiko keamanan informasi, seperti pencurian data, penyadapan komunikasi, dan manipulasi berkas pada jalur transmisi jaringan [1]. Mayoritas layanan penyimpanan data saat ini masih menerapkan mekanisme enkripsi di sisi peladen (*server-side encryption*). Pendekatan ini memiliki kelemahan mendasar

pada aspek kedaulatan privasi, di mana penyedia layanan masih memiliki peluang untuk mengakses data pengguna karena kunci dekripsi berada di bawah kendali peladen pusat, sehingga rentan terhadap kebocoran akibat serangan siber pada database utama [2].

Sebagai alternatif yang lebih tangguh, pendekatan *Client-Side Encryption* terbukti mampu memitigasi risiko kebocoran data tersebut. Pada pendekatan ini, data dalam bentuk aslinya (*plaintext*) dienkripsi langsung pada perangkat pengguna sebelum ditransmisikan, sehingga peladen hanya menerima data acak (*ciphertext*) [2]. Pola ini meminimalkan

ketergantungan keamanan pada pihak ketiga dan melindungi data dari skenario kompromi peladen [2]. Hal ini menjadi fondasi utama dari *Zero-Knowledge Architecture*, yakni sebuah model keamanan di mana penyedia layanan sama sekali tidak memiliki pengetahuan mengenai isi berkas maupun kunci kriptografi yang digunakan oleh pengguna.

Didukung oleh arsitektur web modern, operasi kriptografi kini tidak lagi bergantung pada instalasi aplikasi *desktop* yang berat. Peramban modern telah dilengkapi dengan Web Crypto API, sebuah standar antarmuka bawaan dari W3C yang memungkinkan eksekusi algoritma keamanan tingkat rendah secara langsung di sisi klien dengan memanfaatkan akselerasi perangkat keras [3]. Pemanfaatan API bawaan ini secara signifikan meminimalkan celah keamanan (*cryptographic misuse*) yang sering terjadi akibat kesalahan pengembang saat mengintegrasikan pustaka (*library*) pihak ketiga dari luar [3].

Guna menjawab tantangan privasi tersebut, penelitian ini mengembangkan aplikasi "Secure-Web-Lock". Sistem ini mengintegrasikan algoritma *Advanced Encryption Standard* (AES) dengan mode *Galois/Counter Mode* (GCM) 256-bit, yang sejarah pengembangannya diakui secara global sebagai standar keamanan tingkat tinggi oleh NIST [4]. Mode GCM dipilih karena tidak hanya menjamin kerahasiaan data, tetapi juga secara otomatis memverifikasi integritas berkas melalui pembentukan *Authentication Tag* [2]. Untuk melindungi sistem dari serangan tebakan kata sandi, kunci kriptografi diturunkan menggunakan *Password-Based Key Derivation Function 2* (PBKDF2) dengan fungsi *hash* SHA-256. Guna menyesuaikan dengan standar keamanan industri terkini dalam menahan kecepatan komputasi kartu grafis (GPU) peretas modern, jumlah iterasi PBKDF2 pada sistem ini ditingkatkan menjadi 600.000 iterasi [5]. Penelitian ini bertujuan untuk merancang arsitektur pengamanan berkas yang praktis, memiliki perlindungan *anti-tampering* absolut, dan mengembalikan kendali privasi data sepenuhnya ke tangan pengguna langsung dari peramban web.

Berisi latar belakang atau alasan penelitian; Teori pendukung; Tujuan penelitian; Manfaat hasil penelitian, hipotesis (bila ada).

Daftar pustaka yang digunakan harus relevan dengan konsep penelitian [1].

II. TINJAUAN PUSTAKA

2.1 Kriptografi Simetris dan Advanced Encryption Standard (AES)

Kriptografi simetris menggunakan metode keamanan data yang memanfaatkan satu kunci yang sama untuk mengeksekusi proses enkripsi dan dekripsi. Algoritma simetris yang menjadi standar global saat ini adalah *Advanced Encryption Standard* (AES), yang dikembangkan untuk menggantikan standar lama seperti DES. Berdasarkan analisis komparatif, AES memiliki keunggulan performa komputasi yang tinggi serta resistensi yang kuat terhadap berbagai skenario serangan siber jika dibandingkan dengan algoritma pendahulunya. AES beroperasi menggunakan blok data 128-bit dan mendukung panjang kunci sebesar 128-bit, 192-bit, hingga 256-bit. Tingkat keamanan tertinggi dicapai pada varian AES-256, di mana ruang pencarian kunci mencapai 2^{256} kombinasi, menjadikannya mustahil ditembus melalui serangan tebakan (*brute-force*) dengan kapasitas infrastruktur komputasi saat ini.

2.2 Galois/Counter Mode (GCM)

Dalam implementasinya, algoritma *block cipher* seperti AES memerlukan mode operasi untuk memproses ukuran berkas yang melebihi satu blok data. Proyek ini menerapkan *Galois/Counter Mode* (GCM), sebuah mode operasi yang masuk ke dalam kategori *Authenticated Encryption with Associated Data* (AEAD). AES-GCM bekerja dengan menggabungkan mekanisme *Counter Mode* (CTR) untuk menjamin aspek kerahasiaan (*confidentiality*) dan *Galois Message Authentication Code* (GMAC) untuk menghasilkan *Authentication Tag* sebagai penjamin aspek integritas data. Karakteristik utama AES-GCM adalah kemampuannya mendeteksi perubahan data secara otomatis (*anti-tampering*); jika terdapat manipulasi sekecil satu bit saja pada berkas terenkripsi, proses dekripsi akan langsung digagalkan karena nilai *tag* validasi tidak lagi sesuai.

2.3 Password-Based Key Derivation Function 2 (PBKDF 2)

Kata sandi yang diciptakan oleh manusia umumnya memiliki tingkat keacakan (*entropi*) yang rendah, sehingga sangat berisiko jika langsung ditransformasikan menjadi kunci kriptografi AES. Untuk mengatasi kerentanan tersebut, diperlukan algoritma *key stretching* seperti PBKDF2. PBKDF2 bekerja dengan memperkuat kata sandi dasar melalui penambahan nilai *Salt* acak sepanjang 16-byte guna menangkal serangan *Rainbow Table*. Selanjutnya, kombinasi data tersebut diproses secara berulang menggunakan fungsi *hash* SHA-256. Penggunaan parameter 600.000 iterasi secara sengaja dirancang untuk melambankan waktu komputasi per detik pada perangkat peretas, tanpa mengorbankan kenyamanan pemrosesan lokal di sisi pengguna sah.

2.4 Client-Side Encryption dan Web Crypto API

Client-Side Encryption mengalihkan seluruh beban kalkulasi kriptografi dari peladen pusat ke peramban lokal perangkat klien. Melalui *Web Crypto API*, browser mengeksekusi fungsi-fungsi keamanan tingkat rendah secara *native* memanfaatkan fitur instruksi perangkat keras khusus seperti AES-NI. Pendekatan *native* ini terbukti jauh lebih efisien dibandingkan pustaka berbasis JavaScript murni eksternal. Selain itu, fungsionalitas ini memitigasi kesalahan penulisan kode (*secure coding framework*) oleh pengembang serta melindungi komponen kunci digital di dalam RAM agar bersifat *non-extractable* dari serangan injeksi skrip berbahaya seperti *Cross-Site Scripting* (XSS).

III. METODOLOGI PENELITIAN

3.1 Tahapan Pengembangan Sistem

Sistem dikembangkan menggunakan metode *Prototype* untuk menyelaraskan kebutuhan antarmuka pengguna dengan fungsionalitas fungsi kriptografi secara dinamis. Tahapan pengembangan meliputi analisis kebutuhan fungsional dan non-fungsional, perancangan arsitektur *Zero-Knowledge*, perancangan alur biner kriptografi, implementasi kode program, hingga pengujian fungsionalitas sistem.

3.2 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dirancang untuk memastikan keamanan tingkat tinggi tanpa mengorbankan performa peramban:

- Kebutuhan Fungsional:** Sistem harus mampu memuat berbagai format berkas secara lokal, menerima kata sandi pengguna, melakukan *key stretching* via PBKDF2, mengeksekusi enkripsi/dekripsi berbasis Web Crypto API, serta menyediakan fitur unduh otomatis hasil pemrosesan berkas.
- Kebutuhan Non-Fungsional:** Sistem wajib berjalan sepenuhnya pada sisi klien (*client-side runtime*) tanpa adanya penyimpanan database atau transmisi data sensitif ke peladen luar, serta menerapkan metode pembersihan sisa buffer data di memori RAM setelah proses selesai.

3.3 Perancangan Arsitektur Sistem

Arsitektur Secure-Web-Lock mengimplementasikan pola *Zero-Knowledge Architecture* menggunakan pendekatan *Client-Side Encryption* murni. Peladen web dalam skenario ini hanya bertindak sebagai penyedia aset statis aplikasi (HTML, CSS, dan JavaScript). Ketika aplikasi telah dimuat oleh peramban pengguna, seluruh operasi matematika kriptografi dijalankan secara lokal di dalam memori terisolasi peramban memanfaatkan akselerasi perangkat keras bawaan. Dengan skema ini, dokumen asli (*plaintext*) maupun kata sandi tidak pernah dikirimkan keluar dari perangkat klien, sehingga menutup potensi kebocoran data dari sisi peladen.

3.4 Perancangan Alur Enkripsi

Proses enkripsi berkas dirancang secara terstruktur untuk menghasilkan output biner yang aman dan konsisten.



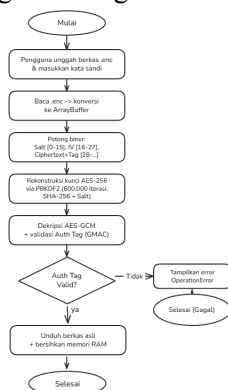
Gambar 1.1 Alur Proses Enkripsi

Tahapan sistematis proses enkripsi digambarkan sebagai berikut:

- Pembangkitan data acak aman: Sistem menghasilkan nilai Salt acak sepanjang 16 byte dan Initialization Vector (IV) acak sepanjang 12 byte menggunakan fungsi generator bilangan acak kriptografis bawaan peramban.
- Derivasi Kunci Kriptografi: Kata sandi tekstual dari pengguna ditransformasikan menjadi kunci simetris AES 256-bit melalui algoritma PBKDF2 dengan basis fungsi hash SHA-256 yang dieksekusi sebanyak 600.000 iterasi.
- Pembacaan Berkas: Berkas masukan dibaca secara asinkron dan dikonversi menjadi struktur data biner ArrayBuffer di dalam memori lokal.
- Enkripsi Data: ArrayBuffer berkas dienkripsi menggunakan kunci AES-GCM dan IV yang telah dibentuk, menghasilkan blok data teracak (ciphertext) bersama dengan Authentication Tag yang otomatis terhitung di akhir data.
- Konstruksi Berkas Akhir: Komponen biner digabungkan ke dalam satu struktur Blob tunggal dengan susunan format: [Salt (16 bytes)] + [IV (12 bytes)] + [Ciphertext + Authentication Tag]. Berkas ini kemudian dikeluarkan dalam bentuk file biner ber-ekstensi .enc.

3.5 Perancangan Alur Dekripsi

Proses dekripsi dirancang dengan skema validasi integritas yang ketat untuk mencegah upaya pembongkaran ilegal.



Gambar 1.2 Alur Proses Dekripsi

Tahapan proses dekripsi adalah sebagai berikut:

- Pemuatan Berkas .enc: Berkas terenkripsi dibaca oleh sistem dan diubah ke bentuk ArrayBuffer.
- Pemotongan Biner (Slicing): Sistem melakukan pemotongan segmen biner secara presisi, di mana 16 byte pertama diisolasi sebagai Salt, 12 byte berikutnya sebagai IV, dan sisa byte ke belakang diidentifikasi sebagai Ciphertext berpasangan dengan Authentication Tag.
- Rekonstruksi Kunci: Kata sandi yang dimasukkan oleh pengguna diproses kembali menggunakan Salt hasil potongan melalui PBKDF2 600.000 iterasi untuk menghasilkan kembali kunci AES 256-bit yang identik.
- Dekripsi dan Validasi Integritas: Web Crypto API mengeksekusi fungsi dekripsi. Komponen Galois Message Authentication Code (GMAC) pada peramban akan mencocokkan nilai Authentication Tag pada berkas dengan kalkulasi runtime. Jika tag valid dan kata sandi benar, data asli (plaintext) akan dikembalikan menjadi dokumen utuh. Jika berkas telah mengalami manipulasi data sekecil satu bit saja (tampering), peramban akan memicu kegagalan sistem otomatis (OperationError) dan memblokir proses rekonstruksi berkas secara absolut.

IV. HASIL DAN PEMBAHASAN

4.1 Implementasi Antarmuka Sistem

Aplikasi Secure-Web-Lock berhasil diimplementasikan sebagai *Single Page Application* (SPA) menggunakan pustaka React.js dan perangkat pemroses Vite. Antarmuka pengguna (UI) dirancang menggunakan *Tailwind CSS* untuk menghasilkan tampilan yang modern, responsif, dan intuitif. Modul utama pada antarmuka meliputi area *drag-and-drop* untuk memuat berkas, kolom input kata sandi yang dilengkapi dengan *Password Strength Meter* interaktif, serta indikator progres visual. Indikator kekuatan kata sandi secara *real-time* mengevaluasi tingkat entropi berdasarkan panjang karakter, kombinasi huruf besar-kecil, angka, dan simbol, guna mengedukasi pengguna

agar tidak menggunakan kunci yang rentan terhadap *dictionary attack*.



Gambar 1.3 Tampilan Secure-Web-Lock

4.2 Pengujian Fungsional

Pengujian fungsionalitas sistem dilakukan menggunakan metode *Black-Box Testing* untuk memvalidasi apakah luaran (*output*) dari operasi kriptografi Web Crypto API sesuai dengan spesifikasi yang dirancang tanpa melihat struktur kode internal. Fokus utama pengujian ini adalah memastikan keberhasilan enkripsi, keberhasilan dekripsi, serta ketahanan sistem terhadap skenario anomali seperti kata sandi tidak valid dan manipulasi berkas (*tampering*). Hasil pengujian dirangkum pada Tabel 1.

Tabel 1. Skenario dan Hasil Pengujian Fungsional Sistem

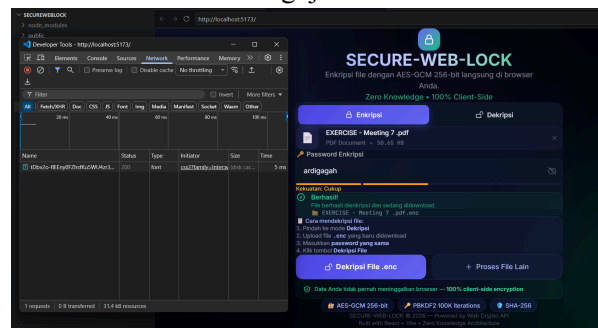
Skenario Pengujian	Aksi yang Dilakukan	Hasil yang Diharapkan	Status
Enkripsi Berkas Normal	Pengguna mengunggah berkas dokumen.pdf, memasukkan kata sandi valid, dan menekan tombol enkripsi.	Sistem menghasilkan dan mengunduh berkas baru bernama dokumen.pdf.enc	Sesuai
Dekripsi Berkas Normal	Pengguna mengunggah berkas .enc, memasukkan kata sandi yang sama persis saat enkripsi.	Sistem mengembalikan berkas ke format asli (dokumen.pdf) dan isinya dapat dibaca sempurna	Sesuai
Dekripsi dengan Kata Sandi Salah	Pengguna mengunggah berkas .enc, namun memasukkan kata sandi yang salah/berbeda	Sistem menolak proses dekripsi, tidak mengunduh berkas apa pun, dan menampilkan	Sesuai

Ketahanan Integritas (*Anti-Tampering*)

Pengguna mengunggah berkas .enc yang telah dimodifikasi (diubah isinya secara paksa via *Hex Editor*), lalu memasukkan kata sandi yang benar.

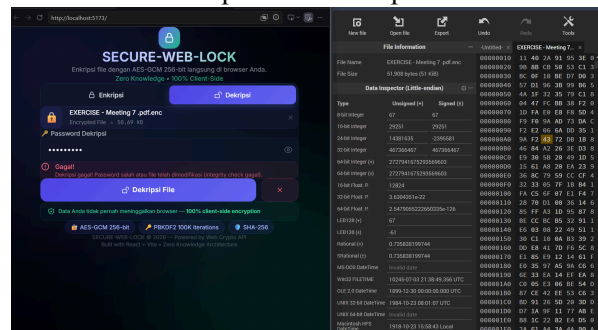
notifikasi *error*. Sistem Sesuai mendeteksi kerusakan segel, menggagalkan proses (Operation error), dan menampilkan peringatan integritas gagal.

4.3 Analisis Hasil Pengujian dan Keamanan



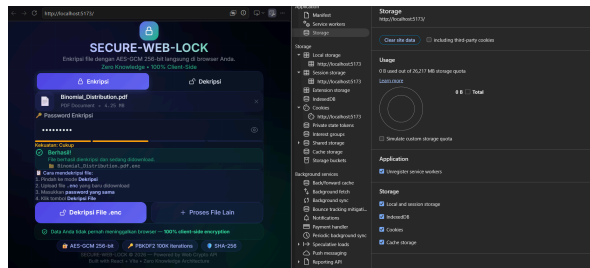
Gambar 1.4 Pembuktian Arsitektur *Zero-Knowledge* melalui Pemantauan Tab *Network* peramban

Gambar 1.4 menunjukkan pemantauan lalu lintas jaringan saat enkripsi berkas berlangsung. Tidak adanya aktivitas pengiriman data (*upload/HTTP POST*) pada tab *Network* membuktikan bahwa komputasi kriptografi dieksekusi murni secara lokal di memori peramban. Hasil ini memvalidasi keberhasilan arsitektur *Zero-Knowledge*, di mana berkas dan kata sandi pengguna terjamin tidak pernah ditransmisikan ke peladen mana pun.



Gambar 1.5 Simulasi Serangan Manipulasi Berkas pada *Hex Editor* (kanan) dan Respons Penolakan Sistem Akibat Kegagalan Validasi Integritas (kanan)

Keunggulan algoritma AES-GCM dalam mendeteksi anomali integritas data diuji melalui skenario manipulasi berkas secara langsung (*tampering*). Pada gambar 1.5, sebuah berkas terenkripsi yang valid dimodifikasi secara ilegal menggunakan *Hex Editor* dengan mengubah nilai satu *byte* secara acak pada area *chiphertext*. Saat berkas modifikasi tersebut diproses kembali oleh sistem menggunakan kata sandi yang valid, peramban secara otomatis memicu pengecualian (*OperationError*). Penolakan ini membuktikan bahwa komponen *Galois Message Authentication Code* (GMAC) pada Web Crypto API berhasil mendeteksi ketidaksesuaian *Authentication Tag*. Sistem secara absolut menggagalkan proses dekripsi, memastikan bahwa pengguna tidak akan pernah menerima atau mengunduh dokumen yang telah disabotase.



Gambar 1.6 Pembuktian Ketiadaan Residu Data Sensitif pada Fasilitas Penyimpanan Peramban Setelah Operasi Kriptografi Selesai

Pengujian keamanan selanjutnya difokuskan pada efisiensi manajemen memori dan mitigasi risiko ekstraksi data (*data dumping*) pada sisi klien. Gambar 1.6 mengilustrasikan hasil inspeksi terhadap fasilitas penyimpanan lokal peramban, yang meliputi *Local Storage*, *Session Storage*, dan *Cookies*, segera setelah proses enkripsi berkas selesai dieksekusi. Hasil inspeksi menunjukkan ketiadaan residu data sensitif, seperti kata sandi asli (*plaintext password*), nilai *Salt*, maupun kunci simetris AES yang telah diturunkan. Hal ini mengonfirmasi bahwa sistem telah mengimplementasikan mekanisme pembersihan variabel memori (*memory wiping*) secara instan pada level DOM (*Document Object Model*). Ketiadaan jejak digital ini secara efektif membentengi aplikasi dari potensi serangan *Cross-Site Scripting* (XSS) maupun eksploitasi akses fisik secara tidak sah, memastikan bahwa peretas tidak dapat mengekstraksi informasi rahasia apa pun meskipun sesi pemrosesan pengguna telah berakhir.

V. KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada aplikasi "Secure-Web-Lock", dapat ditarik beberapa kesimpulan sebagai berikut:

- Aplikasi Secure-Web-Lock berhasil dibangun sebagai sistem pengamanan berkas berbasis web murni menggunakan framework React dan Vite yang berjalan sepenuhnya di sisi klien (*client-side runtime*).
- Penerapan konsep *Zero-Knowledge Architecture* berhasil diwujudkan secara penuh melalui pemanfaatan *Web Crypto API*. Seluruh proses matematis kriptografi (pembangkitan *Salt* dan IV, derivasi kunci, serta enkripsi/dekripsi) dieksekusi secara lokal di dalam peramban pengguna, sehingga berkas asli (*plaintext*) dan kata sandi tidak pernah ditransmisikan ke peladen luar.
- Pengintegrasian algoritma AES-GCM 256-bit terbukti efektif dalam menjaga aspek keamanan data secara komprehensif. Mode GCM tidak hanya menjamin kerahasiaan berkas, tetapi juga memberikan perlindungan *anti-tampering* absolut melalui *Authentication Tag* yang mampu mendeteksi dan menggagalkan rekonstruksi berkas yang telah dimodifikasi secara ilegal sekecil satu bit pun.
- Implementasi fungsi pembentukan kunci PBKDF2 dengan 600.000 iterasi *hash* SHA-256 berhasil meningkatkan resistensi kata sandi pengguna terhadap ancaman serangan tebakan dinamis (*brute-force* dan *dictionary attack*) dengan memanfaatkan parameter penundaan waktu komputasi yang aman bagi pengguna sah.
- Hasil pengujian *Black-Box* menegaskan bahwa sistem memiliki tingkat keandalan dan fungsionalitas yang tinggi

dalam memproses berbagai format berkas digital tanpa mengorbankan aspek portabilitas aplikasi web.

5.2 Saran

Meskipun sistem telah memenuhi target fungsionalitas keamanan yang dirancang, beberapa poin pengembangan berikut dapat dipertimbangkan untuk penelitian selanjutnya:

- a. Mengintegrasikan *Streams* API pada proses pembacaan berkas agar aplikasi mampu menangani enkripsi dan dekripsi berkas berukuran sangat besar (skala *gigabyte*) tanpa mengalami risiko kendala keterbatasan memori RAM (*out-of-memory*) pada peramban klien.
- b. Menambahkan fitur kriptografi asimetris (seperti RSA atau ECC) di atas arsitektur yang sudah ada untuk memfasilitasi mekanisme pertukaran kunci rahasia (*secure key exchange*) antar-pengguna secara aman melalui jaringan publik.
- c. Mengembangkan sistem manajemen visual riwayat enkripsi lokal terenkripsi memanfaatkan penyimpanan internal peramban seperti *IndexedDB* untuk meningkatkan pengalaman pengguna (*user experience*).

DAFTAR PUSTAKA

- [1] “Application of AES-RSA Hybrid Encryption Technology in Data Transmission Practices,” *Journal of Information Engineering and Applications*, Aug. 2025, doi: 10.7176/jiea/15-2-06.
- [2] D. Ramakrishna and M. Ali Shaik, “Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000. A Comprehensive analysis of Cryptographic Algorithms: Evaluating Security, Efficiency, and Future Challenges”, doi: 10.1109/ACCESS.2017.DOI.
- [3] P. Wichmann, M. Blochberger, and H. Federrath, “Web Cryptography API: Prevalence and Possible Developer Mistakes,” in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Aug. 2022. doi: 10.1145/3538969.3538977.
- [4] M. E. Smid, “Development of the advanced encryption standard,” *J. Res. Natl. Inst. Stand. Technol.*, vol. 126, 2021, doi: 10.6028/JRES.126.024.
- [5] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series.